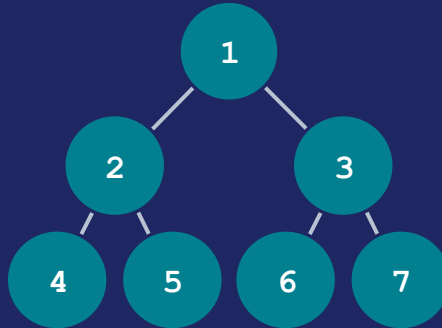


Walking a Tree, Three Ways

Preorder, Inorder and Postorder Traversal

Unit III • Lecture 23



One shape, three orders — the line that moves

preorder

```
visit(root);  
preorder(left);  
preorder(right);
```

inorder

```
inorder(left);  
visit(root);  
inorder(right);
```

postorder

```
postorder(left);  
postorder(right);  
visit(root);
```

It is the SAME three-line function every time — only the position of `visit(root)` moves: first, middle, or last. Memorize the pattern, not three separate functions.

preorder(): visit, then go left, then go right

```
void preorder(struct Node *root)
{
    if (root == NULL)
        return;          /* base case */

    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}
```

- Base case: an empty subtree does nothing and returns — exactly the recursion discipline from Lecture 10.
- The root is printed BEFORE either recursive call — "pre" as in before.
- Used to COPY a tree, or to write it out in a form that can rebuild the same structure (root first tells you where to start).

inorder(): left, then visit, then right

```
void inorder(struct Node *root)
{
    if (root == NULL)
        return;

    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}
```

- The root is printed BETWEEN its two subtrees — "in" as in in-between.
- On a Binary SEARCH Tree (Lecture 24), inorder visits every value in SORTED order — the single most important property of a BST.
- On an expression tree, inorder reproduces the original INFIX expression.

postorder(): left, then right, then visit

```
void postorder(struct Node *root)
{
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}
```

- The root is printed AFTER both subtrees are fully done — "post" as in after.
- This is EXACTLY Lecture 22's freeTree — free children before freeing yourself, or you would free a node while still needing to reach its children through it.
- On an expression tree, postorder gives POSTFIX — safe to evaluate with a stack (Lecture 10).

Complexity: every node, exactly once

Time

$O(n)$ for all three.

Each call visits one node and makes exactly two recursive calls — no node is ever revisited.

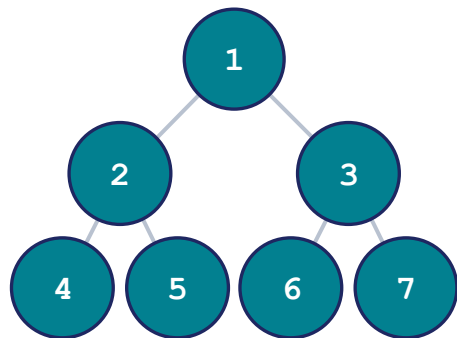
Space (call stack)

$O(h)$, where h is the tree's HEIGHT (Lecture 22).

A skewed tree of n nodes has $h = n-1$ — recursion depth n , same risk as unbounded recursion from Lecture 10.

- A COMPLETE tree of n nodes has $h \approx \log_2 n$ — shallow recursion, small stack.
- The SAME n nodes, skewed, has $h = n-1$ — deep recursion, one frame per node.
- Same node count, wildly different stack cost — shape matters here exactly as it did for array storage in Lecture 22.

All three traversals, one tree, verified



preorder(root)

1 2 4 5 3 6 7

inorder(root)

4 2 5 1 6 3 7

postorder(root)

4 5 2 6 7 3 1

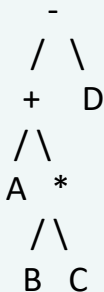
Compiled and run — not hand-invented. Node count: 7, tree height: 2.

What an expression tree actually is

An expression tree is a binary tree where the two node types have fixed jobs:

- **Leaves** hold operands — numbers or variables (A, B, 5, x).
- **Internal nodes** hold operators (+, −, ×, /), and each operator's **two children are its two operands** — which might themselves be whole sub-expressions.

A + B × C − D:



Callback: the expression tree, now with real code

```
for A + B × C - D
```

```
preorder(root) = - + A * B C D    (prefix)
```

```
inorder(root) = A + B * C - D    (infix)
```

```
postorder(root) = A B C * + D -    (postfix)
```

One function, three positions for visit()

- preorder: visit, left, right — root first (copying, prefix expressions).
- inorder: left, visit, right — root between (sorted output on a BST, infix expressions).
- postorder: left, right, visit — root last (the only safe order to free/delete a tree, postfix expressions).
- All three: $O(n)$ time (every node once), $O(h)$ stack space — shape (Lecture 22) determines whether that stack stays shallow or grows to n .

EXIT TICKET For the tree with root P, left child Q, right child R (Q and R are leaves), write all three traversal outputs by hand.